Chapter 4 (Sections 3–4): Local Search Algorithms (Exam Notes)

0. Notation and Definitions

To analyze these algorithms, we define the following mathematical symbols and terms:

- s or x: Represents a **State**. This is a complete configuration of the problem (e.g., in n-queens, a board where all n queens are already placed).
- $f(s)$ or $VALUE(s)$: The **Objective Function**. A mapping that assigns a numeric value to a state. In maximization, a higher value is better.
- $N(s)$: The **Neighborhood** of state s. The set of all states reachable from s by a single modification (e.g., moving one queen in its column).
- ΔE (Delta Energy): The change in value between a candidate state and the current state:

$$\Delta E = VALUE(next) - VALUE(current).$$

- T (Temperature): A positive numeric parameter that controls the probability of accepting "downhill" steps.
- e: Euler's number (≈ 2.718), used in the expression $e^{\Delta E/T}$.
- t: A **time step** or iteration counter (e.g., $t = 0, 1, 2, \dots$).
- $T(t)$: A **cooling schedule**, meaning the value of the temperature at iteration t.
- ∇f (Gradient): A vector of partial derivatives pointing in the direction of the steepest increase in f.
- H (Hessian Matrix): A matrix of second-order partial derivatives describing the local curvature of the function.

1. Iterative Improvement and Local Search

In many optimization problems, the **path** to the goal is irrelevant; only the goal state itself is the solution.

- **State Space**: The set of "complete" configurations.
- **Complete-state formulation**: Every state has all components present, and we move from state to state to improve the configuration.
- **Memory**: These algorithms often use **constant space**, since they keep only the current state (or a small set of states).

2. Hill-Climbing (Gradient Ascent)

Hill-climbing continually moves to a neighbor with higher value ("uphill") and stops when no neighbor improves the current state.

The Algorithm

1. Start with a current state.
2. Select a neighbor with the **highest value**.
3. If $VALUE(neighbor) \leq VALUE(current)$, return the current state.
4. Otherwise, set the neighbor as the current state and repeat.

Landscape Features and Failures

- **Local Maximum**: Higher than all its neighbors but not globally best; hill-climbing gets stuck.
- **Plateau**: Flat region of equal value; progress can stall.
- **Random-Restart Hill Climbing**: Repeat hill-climbing from many random initial states; success improves with restarts.

3. Simulated Annealing (Expanded Explanation)

Simulated annealing is a local search method that attempts to avoid getting trapped in local maxima by sometimes accepting moves that **reduce** the objective value. It combines:

- **Exploration** early (many non-improving moves allowed).
- **Exploitation** late (mostly improving moves allowed).

3.1 Core Idea and Why It Is Needed

Hill-climbing fails in landscapes that contain local maxima and plateaus. If the current state is a local maximum, then every neighbor is worse:

$$\forall s' \in N(s), \quad VALUE(s') \leq VALUE(s).$$

Hill-climbing stops immediately. Simulated annealing addresses this by permitting "downhill" moves with a probability that depends on:

- how bad the move is (how negative ΔE is), and
- how high the temperature T is at that time.

3.2 Decision Rule (Acceptance Rule)

At each step, the algorithm selects a **random** successor state $next$ from the neighborhood $N(current)$ and computes:

$$\Delta E = VALUE(next) - VALUE(current).$$

Then:

1. If $\Delta E > 0$, accept the move (because it improves the value).
2. If $\Delta E \leq 0$, accept the move **with probability**

$$P(\text{accept}) = e^{\Delta E/T}.$$

3.3 Meaning of the Exponential Probability

Assume a maximization problem. For a bad move, $\Delta E < 0$.

- Since $\Delta E/T$ is negative, $e^{\Delta E/T}$ is between 0 and 1, so it is a valid probability.
- **Effect of ΔE:** If a move is much worse (more negative ΔE), then $\Delta E/T$ is more negative, and the acceptance probability becomes smaller.
- **Effect of T:** If T is large, then $\Delta E/T$ is closer to 0, and $e^{\Delta E/T}$ becomes closer to 1, meaning even bad moves are often accepted.

Two limiting behaviors are important:

- **Very high temperature:** If T is very large, then $\Delta E/T \approx 0$, so

$$e^{\Delta E/T} \approx e^0 = 1,$$

and many bad moves are accepted (high exploration).

- **Very low temperature:** If T is close to 0, then for a bad move $\Delta E/T$ is a large negative number, so

$$e^{\Delta E/T} \approx 0,$$

and almost no bad moves are accepted (behavior approaches hill-climbing).

3.4 Cooling Schedule $T(t)$

A **cooling schedule** is a rule that decreases T over time:

$$T(0) > T(1) > T(2) > \cdots$$

The schedule controls the tradeoff:

- If cooling is too fast, the algorithm becomes greedy too soon and can still get trapped (similar to hill-climbing).
- If cooling is very slow, the algorithm explores thoroughly but may take too long to finish.

3.5 Numerical Example of a Bad Move (Using $T = 100, 20, 0.5$)

Suppose we are maximizing and:

$$VALUE(current) = 50, \quad VALUE(next) = 45.$$

Then:

$$\Delta E = 45 - 50 = -5.$$

Case A: Very High Temperature ($T = 100$)

$$P = e^{-5/100} = e^{-0.05} \approx 0.9512.$$

Interpretation: About 95.12% chance to accept this bad move (very exploratory).

Case B: Medium Temperature ($T = 20$)

$$P = e^{-5/20} = e^{-0.25} \approx 0.7788.$$

Interpretation: About 77.88% chance to accept this bad move (still exploratory, but less than at $T = 100$).

Case C: Low Temperature ($T = 0.5$)

$$P = e^{-5/0.5} = e^{-10} \approx 0.0000454.$$

Interpretation: About 0.00454% chance (almost never accepts bad moves; nearly hill-climbing behavior).

3.6 How Simulated Annealing Escapes Local Maxima

If the current state is a local maximum, moving to any neighbor requires accepting a bad move (negative ΔE). At higher temperatures, the acceptance probability can be large enough that the algorithm can move away from the local maximum. Once it leaves the local peak, it may later move into a different region of the landscape that leads to a higher maximum. The essential mechanism is:

- accept some downhill moves early to cross "barriers",
- then gradually reduce acceptance of downhill moves to stabilize near a good solution.

3.7 Optimality: When It Achieves It and Why It Is Hard in Practice

Theoretical optimality statement (idealized). Simulated annealing can achieve global optimality if the temperature is decreased **sufficiently slowly**. In the ideal case:

- the schedule $T(t)$ goes to 0 slowly enough, and
- the algorithm is allowed to run for a very long time (conceptually unbounded time),

then the probability that the algorithm eventually reaches a globally optimal state approaches 1.

Where the optimality is achieved. The global-optimality guarantee is asymptotic: it is achieved in the limit of extremely slow cooling and long runtime, as the temperature gradually approaches 0.

Why it is nearly not possible in practical life. In real applications, the theoretical conditions are difficult to satisfy:

- **Cooling must be extremely slow:** The schedules that support theoretical guarantees can require a very large number of iterations. This can be impractical when each step is expensive.
- **Finite time and finite computation:** Real systems must stop after a limited number of steps. Stopping early weakens the theoretical guarantee.
- **Unknown landscape:** The schedule that is "slow enough" depends on properties of the objective landscape (such as barrier heights), which are usually unknown.
- **Cost of evaluating neighbors:** Even if acceptance is probabilistic, each step still requires generating and evaluating candidate successors; for large problems this is expensive.

Therefore, in practice, simulated annealing is used as a strong heuristic method: it often finds very good solutions, but its strict theoretical optimality is typically not realized under realistic time constraints.

4. Local Beam Search

This algorithm keeps track of k states rather than one.

- **Process**: Generate all successors of all k states; select the **top** k among them.
- **Recruitment**: Good regions can dominate because many top successors may come from the same strong state.
- **Stochastic Beam Search**: Randomized selection biased toward good successors helps maintain diversity.

5. Genetic Algorithms (Introductory Explanation)

Genetic algorithms (GAs) are inspired by biological evolution and use a population of candidate solutions encoded as strings. They evolve the population over time using selection, crossover, and mutation.

- **Population**: A set of candidate solutions.
- **Selection**: Individuals are selected probabilistically based on fitness (objective value).
- **Crossover**: Pairs of individuals exchange parts of their strings at a random crossover point to create offspring.
- **Mutation**: Random changes are applied to offspring to maintain diversity.

Note: Genetic algorithms will be studied in detail in a later chapter. This is a brief overview based on the current material.

6. Local Search in Continuous State Spaces

Many real-world problems involve continuous variables rather than discrete ones. For example, siting three airports in Romania involves a 6-dimensional state space defined by $(x_1, y_1, x_2, y_2, x_3, y_3)$.

6.1 Objective Function Example

In the airport example, the objective function f could be the sum of squared distances from each city to its nearest airport. Locally, this might look like:

$$f(x_1, y_1, \dots) = (x_1 - x_{Arad})^2 + (y_1 - y_{Arad})^2 + \dots$$

6.2 Discretization Methods

One way to handle continuous spaces is to turn them into discrete ones.

- **Empirical Gradient**: Consider a small change $\pm\delta$ in each coordinate to see which direction improves the value.

6.3 Gradient Methods

If the function is differentiable, we can use the **gradient** ∇f, which is the vector of partial derivatives:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

For the airport example, the partial derivative with respect to x_1 would be:

$$\frac{\partial f}{\partial x_1} = 2(x_1 - x_{Arad}) + 2(x_1 - x_{Sibiu}) + \dots$$

- **Update Rule**: $x \leftarrow x + \alpha \nabla f(x)$, where α is the step size.
- **Exact Solution**: Sometimes we can solve $\nabla f(x) = 0$ exactly (e.g., with only one airport).

6.4 Newton-Raphson Method

The Newton-Raphson method (1664, 1690) is an iterative technique to solve $\nabla f(x) = 0$. It uses the **Hessian matrix** H, where $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$.

- **Update Rule**: $x \leftarrow x - H_f^{-1}(x) \nabla f(x)$.
- This method uses second-order information to find the peak more efficiently but requires inverting the Hessian.

7. Summary Comparison Table

Algorithm	**Strategy**	**Memory**	**Optimality**
Hill-Climbing	Greedy improvement	$O(1)$	Local optima only
Simulated Annealing	Probabilistic downhill + cooling	$O(1)$	Global (in theory, with slow cooling)
Local Beam Search	Keep best k states	$O(k)$	Depends on k and diversity
Genetic Algorithms	Population-based crossover + mutation	$O(pop)$	Stochastic global (heuristic)
Gradient Methods (Continuous)	Use derivatives to ascend	$O(1)$	Local optima only
Newton-Raphson (Continuous)	Use second derivatives	$O(1)$	Local optima only